System Architecture

Modules, layers, and data flow

Project: AI_ML_GENAI

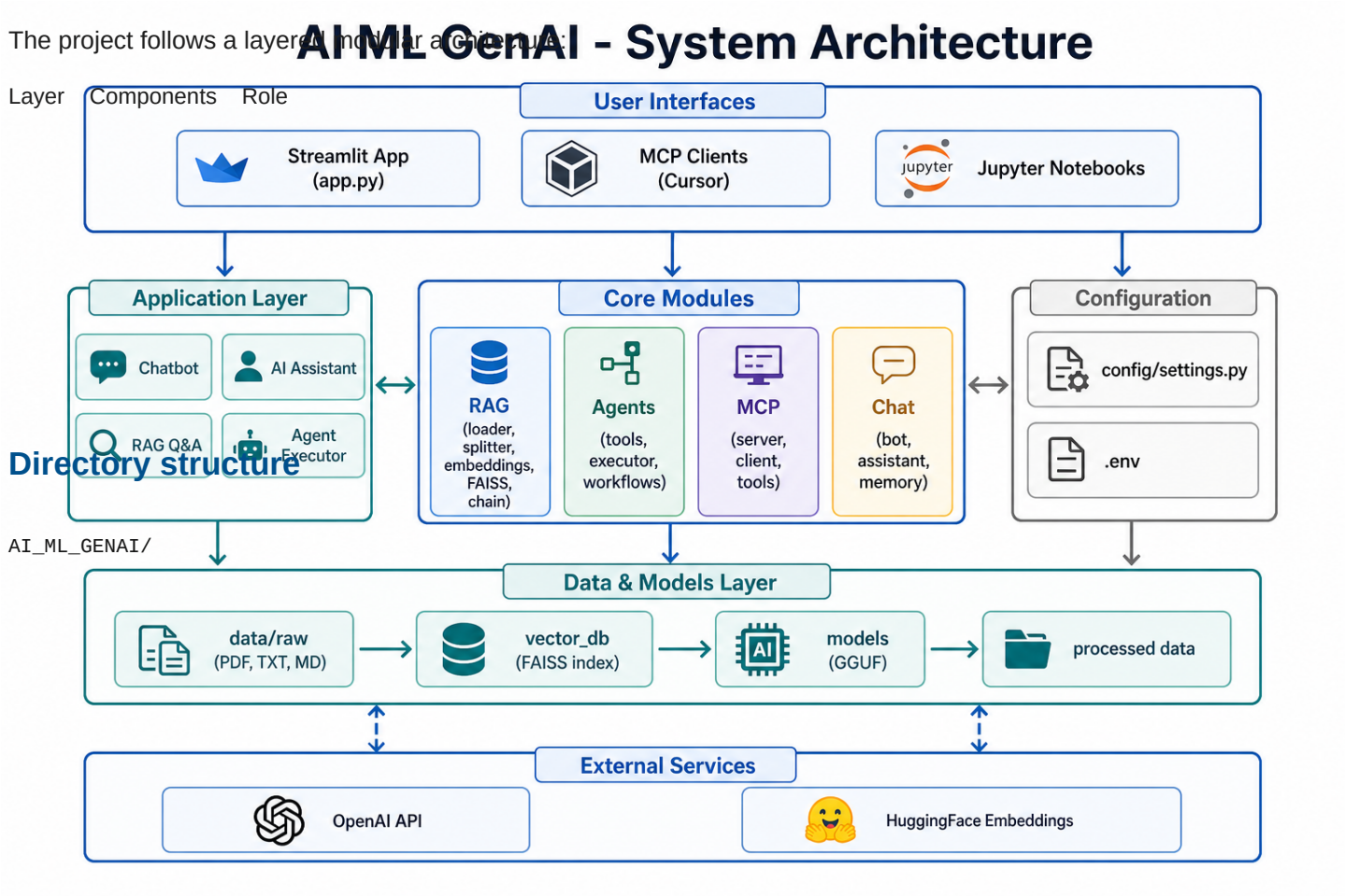
Architecture

System design and module responsibilities for the AI / ML / GenAI workspace.

High-level architecture

System architecture overview

The project follows a layered modular architecture:



Layer	Components	Role

Directory structure

```
AI_ML_GENAI/
```

Module: config/

File: config/settings.py

Central configuration loaded from environment variables via python-dotenv.

Setting	Purpose

All modules import from here - no hardcoded paths in business logic.

Module: rag/

Retrieval-Augmented Generation pipeline.

File Responsibility

------ -

Design choice: RAG is isolated so it can be used by app.py, chat/assistant.py, and future APIs without duplication.

Module: agents/

Tool-calling AI agents.

File Responsibility

------ -

Design choice: Agents use LangChain's create_tool_calling_agent pattern. Tools are registered in one place for easy extension.

Module: mcp_server/

Model Context Protocol integration.

File Responsibility

------ -

Design choice: MCP runs as a separate process, enabling integration with Cursor and other MCP-compatible clients without modifying the main app.

Module: chat/

Conversational interfaces.

File Responsibility

Design choice: Chat and RAG are composable - AIAssistant optionally loads the RAG chain when vector_db/ exists.

Data flow summary

Flow Path

See Flow Diagrams for detailed step-by-step flows.

Technology stack

Component Technology

Extension points

Want to add... Where to start

High-Level System Architecture